J MUKHESH

2303A51813

**98.check if a binary tree is binary search tree**

```
class Solution {
  public boolean isValidBST(TreeNode root) {
    List<Integer> ans=new ArrayList<>();
    inorder(ans,root);
    for(int i=1;i<ans.size();i++){
      if(ans.get(i-1)>=ans.get(i)) return false;
    }
    return true;
  }
  public void inorder(List ans,TreeNode root){
    if(root==null) return ;
    inorder(ans,root.left);
    ans.add(root.val);
    inorder(ans,root.right);
  }
}
```

**230 kth smallest element in BST**

```
class Solution {
  public int kthSmallest(TreeNode root, int k) {
    Queue<TreeNode> q = new LinkedList<>();
    List<Integer> list = new ArrayList<>();
    q.add(root);
    while(!q.isEmpty()){
      int n = q.size();
```

```
        for(int i = 0; i < n; i++){
            TreeNode temp = q.poll();
            list.add(temp.val);
            if(temp.left != null)
                q.add(temp.left);

            if(temp.right != null)
                q.add(temp.right);
        }
    }
    Collections.sort(list);
    return list.get(k-1);
  }
}
```

**110 balanced binary tree**

```
class Solution {
  public int height(TreeNode root){
    if(root == null) return 0;
    int left = height(root.left);
    if(left == -1) return -1;
    int right = height(root.right);
    if(right == -1) return -1;
    if(Math.abs(left-right)>1) return -1;
     return 1+Math.max(left,right);

  }
  public boolean isBalanced(TreeNode root) {
    return height(root)!=-1;
```

```
    }
}
```